4 Documenting Requirements

In requirements engineering, information that has been established or worked out during different activities must be documented. Among this information are, for example, protocols of interviews and reports of validation or agreement activities, but also change requests. The main and most important documentation task in requirements engineering, though, is to document the requirements for the system in a suitable manner.

4.1 Document Design

A documentation technique is any kind of more or less formal depiction that eases communication between stakeholders and increases the quality of the documented requirements. In principle, any kind of documentation technique can be used to document the requirements, let it be natural language-based documentation by means of prose, more structured natural language-based text, or more formal techniques such as state diagrams.

Definition 4-1: *Requirements Document / Requirements Specification*
A requirements specification is a systematically represented collection of requirements, typically for a system or component, that satisfies given criteria.

Reasons for the documentation

During the life cycle of a requirements document, many people are trusted with the documentation. During communication, the documentation has a goal-oriented and supporting role. The main reasons for documenting requirements are as follows:

Central role of requirements

- *Requirements are the basis of the system development*. Requirements of any kind influence the analysis, design, implementation, and test phases directly and indirectly. The quality of a requirement or of a requirements document has a strong impact on the progress of the project and therefore on its success.

Legal relevance

- *Requirements have a legal relevance*. Requirements are legally binding for the contractor and the client, and the client can sue for their fulfillment. Documenting the requirements can help to quickly overcome legal conflicts between two or more parties.

Complexity

- *Requirements documents are complex*. Systems that possess thousands of requirements that in turn have complex interdependencies on multiple layers are not unheard of in practice. Without suitable documentation, keeping on top of things can become very difficult for anyone involved.

Accessibility

- *Requirements must be accessible to all involved parties*. Projects undergo certain "development" as time goes by—with regard to the subject as well as the staff. When requirements can be permanently accessed, uncertainty and obscurities can be avoided and staff that has recently joined the project can quickly get up to speed.

Another argument for a good documentation, supportive of the project, is that employees almost never share the same understanding of a subject matter. Therefore, requirements should be documented in a way that they meet the quality demands of all involved.

4.2 Types of Documentation

Requirements for a system can be documented in three different perspectives. In practice, natural language as well as conceptual models are used to this end, or

oftentimes, an advantageous combination of both is employed.

4.2.1 The Three Perspectives of Requirements

Requirements for a system can be documented in three different perspectives onto the system to be developed:

Data perspective

- *Data perspective:* In the data perspective, a static-structural perspective on the requirements of the system is adopted. For example, the structure of input and output data as well as static-structural aspects of usage and dependency relations of the system and the system context can be documented (e.g., the services of an external system).

Functional perspective

- *Functional perspective:* The functional perspective documents which information (data) is received from the system context and manipulated by the system or one of its functions. This perspective also documents which data flows back into the system context. The order in which functions processing the input data are executed is also documented.

Behavioral perspective:

- *Behavioral perspective:* In the behavioral perspective, information about the system and how it is embedded into the system context is documented in a state-oriented manner. This is done by documenting the reactions of the system upon events in the system context, the conditions that warrant a state transition, and the effects that the system shall have on its environment (e.g., effects of the system analyzed that represent events in the system context of a different system).

4.2.2 Requirements Documentation using Natural Language

Advantages of using natural language

Natural language, particularly prose, is the most commonly applied documentation form for requirements in practice. In contrast to other documentation forms, prose has a striking advantage: No stakeholder has to learn a new notation. In addition, language can be used for miscellaneous purposes—the requirements engineer can use natural language to express any kind of requirement.

Disadvantages of using natural language

Natural-language-based documentation is well suited to document requirements in any of the three perspectives. However, natural language can allow requirements to be ambiguous, and requirements of different types and perspectives are in danger of being unintentionally mixed up during documentation. In that case, it is difficult to isolate information pertaining to a certain perspective amidst all of the requirements in natural language.

4.2.3 Requirements Documentation using Conceptual Models

In contrast to natural language, the different types of conceptual models cannot be used universally. When documenting requirements by means of models, special modeling languages must be used that pertain to the appropriate perspective. Assuming the modeling language selected for a documentation task is applied correctly, its use constructively guarantees that the models created depict information pertaining to the respective perspective only. The models depict the documented requirements much more compactly and they therefore are easier for a trained reader to understand than is natural language. In addition, conceptual models offer a decreased degree of ambiguity (i.e., fewer ways to be interpreted) than natural language due to their higher degree of formality. However, using conceptual modeling languages for requirements documentation requires specific knowledge of modeling. The following list includes short descriptions of the most important diagrams discussed in chapter 6.

Overview of system functions

- *Use case diagram:* A use case diagram allows you to gain a quick overview of the functionalities of the specified system. A use case describes which functions are offered to the user by the system and how these functions relate to other external interacting entities. However, use cases do not describe the

responsibilities that the functions have in detail (see section 6.3).

Structural data modeling and structuring of terms

- *Class diagram:* Among other things, class diagrams are used in requirements engineering to document requirements with regard to the static structure of data, to document static-structural dependencies between the system and the system context, or to document complex domain terms in a structured manner (see section 6.5.2).

Sequence modeling

- *Activity diagram:* Using activity diagrams, business processes, or sequence-oriented dependencies of the system in regard to processes within the system context can be documented. Activity diagrams are also well suited to model the sequential character of use cases or to model a detailed specification of the interaction of functions that process data (see section 6.6.3).

Event-driven behavior

- *State diagram:* State diagrams are used in requirements engineering to document event-driven behavior of a system. The focus of this type of model is on the individual states the system can be in, events and their respective conditions that trigger a state transition, and effects of the system in its environment.

4.2.4 Hybrid Requirements Documents

Combined use of documentation types

Requirements documents first and foremost contain requirements. In addition, in many situations it is sensible to document decisions, important explanations, and other relevant information as well. Depending on the target audience of the document, the perspective on the system, and the documented knowledge, suitable documentation types are selected. Typically, documents contain a combination of natural language and conceptual models. The combination allows the disadvantages of both documentation types to be decreased by means of the

strengths of the other documentation type, and combining documentation types exploits the advantages of both. For instance, models can be amended or complemented by natural language comments and natural language requirements and natural language glossaries can be summarized and their dependencies can be depicted clearly by making use of models.

4.3 Document Structures

Influence of the requirements on satisfaction

Requirements documents contain a magnitude of different information. These must be well structured for the reader. In order to do that, one can make use of standardized document structures or individually define a custom document structure.

4.3.1 Standardized Document Structures

Adaptation of existing standard outlines

Standard outlines offer a predefined structure, i.e., predefined stereotypes according to which the information can be classified. By using standard outlines, a rough structure along with a short description of the content of the main sections is predetermined. Using standard outlines has the following advantages:

- Standard outlines simplify incorporating new staff members.
- Standard outlines allow for quickly finding desired contents.
- Standard outlines allow for selective reading and validation of requirements documents.
- Standard outlines allow for automatic verification of requirements documents (e.g., with regard to completeness).
- Standard outlines allow for simplified reuse of the contents of requirements documents.

It must be noted that these structures must be tailored with regard to the specific project properties to meet the respective constraints. In the following paragraphs, three of the most widely used standardized document structures are introduced.

Rational Unified Process

The *Rational Unified Process (RUP)* [Kruchten 2001]is usually used for software systems that are developed using object-oriented methods. The client creates a *business model* that contains different artifacts from the business environment (e.g., business rules, business use cases, business goals), which serve as the basis for requirements of the system over the course of development. The contractor uses the structures of the *software requirements specification (SRS)* to document all software requirements. These structures are closely related to the ISO/IEC/IEEE standard 29148:2011, as described next.

ISO/IEC/IEEE standard 29148:2011

The ISO/IEC/IEEE standard 29148:2011 [ISO/IEC/IEEE 29148:2011] contains an outline designed for the documentation of software requirements (software requirements specification). The standard structure suggests dividing the requirements document into five parts with regard to their subject matter:

- A chapter with introductory information (e.g., system goal, system bounding) and a general description of the software (e.g., perspective of the system, properties of future users)
- A chapter with a listing of all documents that are referenced in the specification
- A chapter for specific requirements (e.g., functional requirements, performance, interfaces)
- A chapter with all planned measures for verification
- Appendices (e.g., information about assumptions that were made, identified dependencies)

V-Model

The *V-Model* [V-Modell 2004] of the German Federal Ministry of the Interior (BMI) defines different structures, depending on the creator of the requirements document:

- The *Customer Requirements Specification*, known in the German original as Lastenheft, is created by the customer and describes all of the demands to the contractor regarding the subject of the contract, i.e., deliveries and services.

In addition, in many cases, demands of the users, including all constraints to the system and the development process, are documented. Therefore, the Customer Requirements Specification usually describes *what* is made *for what*.

- The *System Requirements Specification*, known in the German original as Pflichtenheft, is based on the Customer Requirements Specification and contains the implementation suggestions that the contractor has elaborated. Therefore, the System Requirements Specification is a refinement of the requirements and constraints of the Customer Requirements Specification.

4.3.2 Customized Standard Contents

The minimum content

As described in section 4.3.1, standardized document structures are adapted with regard to the specific project conditions. The following issues should be addressed by any chosen structure.

Introduction

The introduction contains information about the entire document. This information allows gaining a quick overview of the system.

- *Purpose:* This section discusses why the document was created and who the target audience for the requirements document is.
- *System coverage:* This part consists of the system to be developed. It indicates system name and the principle goals and advantages that arise from introducing the system.
- *Stakeholder:* This section contains a list of stakeholders and their relevant information (see section 3.1.1).
- *Definitions, Acronyms, and Abbreviations:*[1] In this section, the terms used in the document are defined so that they can be used consistently throughout the document.
- *References:*[2] All documents that are referenced by the requirements document are listed herein.
- *Overview:* At the end of the introductory chapter, the content and structure of

the following sections of the requirements document should be explained briefly.

General Overview

In this section, additional information is documented that increases the understandability of the requirements. In contrast to the introduction, this is merely operational information that does not pertain to administration, management, or organizational aspects of the requirements document.

- *System environment:* The embedding of the system into the environment is of key concern in this paragraph. The results of your definition of the system boundary and context boundary can be found herein.
- *Architecture description:* In this section, the operational interfaces of the system (e.g., user interfaces, hardware and software interfaces, and communication interfaces) are documented. In addition, further information, e.g., regarding storage limitations, is also discussed.
- *System functionality:* This section contains the coarse functionalities and tasks of the system. This can be documented, for example, using a use case diagram.
- *User and target audience:* The different users of the system that make up the target audience are listed.
- *Constraints:* In this section, all conditions ought to be listed that have not been documented thus far and might hinder the requirements engineering.
- *Assumptions:* Decisions, such as not implementing certain aspects of the system due to budgeting reasons, or other general assumptions about the system context that the requirements are based upon are documented here.

Requirements

This part contains functional requirements as well as quality requirements.

Appendices

In the appendices, additional information that completes the document can be documented. For example, the appendices can include additional documents

regarding the user characteristics, standards, conventions, or background information regarding the requirements document.

Index

The index typically contains a table of contents (i.e., a structure of the chapters) and an index directory. In highly dynamic requirements documents, this may be a highly critical section that must be kept up-to-date.

4.4 Using Requirements Documents

Requirements documents as the basis for development

Over the course of the project, requirements documents serve as the basis for different tasks:

- *Planning:* Based on the requirements document, concrete work packages and milestones for the implementation of the system can be defined.
- *Architectural design:* The detailed documented requirements (along with constraints) serve as the basis for the design of the system architecture.
- *Implementation:* Based on the architectural design, the system is implemented by making use of the requirements.
- *Test:* On the basis of requirements that have been documented in the requirements document, test cases can be developed that can be used for system validation later on.
- *Change management:* When requirements change, the requirements document can serve as the basis to analyze the extent to which other parts of the system are influenced. The change effort can thus be estimated.
- *System usage and system maintenance:* After the system is developed, the requirements document is used for maintenance and support. This way, the requirements document can be used to analyze concrete defects and shortcomings that surface during system use. For example, one can deduct if a defect is a result of using the system incorrectly, a result of an error in requirements, or a result of an error in implementation.
- *Contract management:* The requirements document is the prime subject of a contract between a client and a contractor in many cases.

4.5 Quality Criteria for Requirements Documents

To become a basis for the subsequent processes, the requirements document must meet certain quality criteria. According to the ISO/IEC/IEEE standard 29148:2011 [ISO/IEC/IEEE 29148:2011], a requirements document shall be complete and consistent. Moreover, a requirements document shall support readability by offering a clear structure, reasonable scope, and traceability. Overall, a requirements document shall fulfil the following quality criteria:

- Unambiguity and consistency
- Clear structure
- Modifiability and extendibility
- Completeness
- Traceability

4.5.1 Unambiguity and Consistency

Quality of individual requirements is a prerequisite.

Requirements documents can be consistent and unambiguous only when the individual requirements are consistent and unambiguous. In addition, it must be guaranteed that individual requirements do not contradict one another. To achieve this, it is advisable to make use of conceptual models (see chapter 6). Another aspect of unambiguity pertains to the unique identification of a requirements document or a requirement among the set of all requirements documents or requirements in a development project (see section 8.5).

4.5.2 Clear Structure

Allows for selective reading

In order to guarantee that the requirements document is readable by any stakeholder, it should be appropriately comprehensive and clearly structured. Unfortunately, no clear-cut suggestions can be made regarding the appropriate comprehensiveness of a requirements document. A very comprehensive

requirements document with a good structure can be just as appropriate as a less comprehensive document because a clear structure will allow the reader to skip parts that are not relevant to him. An unstructured or badly structured requirements document of the same high level of comprehensiveness would not be appropriate because the document must be read in its entirety in order for a stakeholder to be able to identify parts that are relevant to her. A good starting point is the standard structures described in section 4.3.1.

4.5.3 Modifiability and Extendibility

Content and structure should support changeability.

Requirements documents must be easy to extend. There are always requirements that are changed, altered, added, or removed as a project progresses. As a result, the structure of requirements documents should be easy to modify and extend. The requirements documents of a project should be subject to the project's version control management.

4.5.4 Completeness

Two types of completeness in requirements documents

Requirements documents must be complete, i.e., they must contain all relevant requirements (and required additional information), and each requirement must be documented completely.[3] All possible inputs, influential factors, and required reactions of the system must be described for each desired system function. This comprises describing error and exception cases in particular. Also, quality requirements, such as requirements pertaining to reaction times or availability and usability of the system, must be noted.

Evidence, reference, and sources are formal necessities.

Formal factors also contribute to completeness. Graphs, diagrams, and tables should be appropriately labeled. Another important aspect is that consistent reference and index directories must exist. Also, definitions and norm reference that denote specific terms must be included in any requirements document. The

comprehensiveness of a requirements document is a challenge during requirements engineering. Often, a compromise must be found between the time resources available and the completeness of the requirements documents.

4.5.5 Traceability

Relationship to other development documents

An important quality criterion is traceability of relationships between requirements documents and other documents (e.g., business process model, test plans, or design plans). These documents could have been created in previous development phases, in subsequent development phases, or concurrently with the requirements documents. Among other things, traceability supports change management (see section 8.4).

4.6 Quality Criteria for Requirements

Quality criteria for single document requirements

Each documented requirement should fulfil the following quality criteria:

- *Agreed*: A requirement is agreed upon if it is correct and necessary in the opinion of all stakeholders.
- *Unambiguous*: [ISO/IEC/IEEE 29148:2011] A requirement that is unambiguously documented can be understood in only one way. It must not be possible to interpret the requirement in a different way. All readers of the requirement must arrive at the same understanding of the requirement.
- *Necessary*: [ISO/IEC/IEEE 29148:2011] A documented requirement must represent the facts and conditions of the system context in a way that it is valid with regard to the actualities of the system context. These actualities may be the different stakeholders' ideas, relevant standards, or interfaces to external systems.
- *Consistent*: [ISO/IEC/IEEE 29148:2011] Requirements must be consistent with regard to all other requirements, i.e., the requirements must not contradict one another, regardless of their level of detail or documentation type. In addition, a requirement must be formulated in a way that allows for

consistency with itself, i.e., the requirement may not contradict itself.

- *Verifiable*: [ISO/IEC/IEEE 29148:2011] A requirement must be described in a way that allows for verification. That means that tests or measurements can be carried out that provide evidence of the functionality demanded by the requirement.
- *Feasible*: [ISO/IEC/IEEE 29148:2011] It must be possible to implement each requirement given the organizational, legal, technical, or financial constraints. This means that a member of the development team ought to be involved in rating the goals and requirements so that he can show the technical limits of the implementation of a particular requirement. In addition, the costs for the implementation must be incorporated into the rating. Occasionally, stakeholders withdraw a requirement if the costs for its realization become apparent.
- *Traceable*: [ISO/IEC/IEEE 29148:2011] A requirement is traceable if its origin as well as its realization and its relation to other documents can be retraced. This can be done by means of unique requirement identifiers. Using these unique identifiers, requirements that are derived from other requirements on a different level of the specification can be connected. For example, a system goal can be traced through all levels of abstraction, from design to implementation and test. Details can be found in section 8.4.
- *Complete*: [ISO/IEC/IEEE 29148:2011] Each individual requirement must completely describe the functionality it specifies. Requirements that are yet incomplete must be specially marked, for example by inserting “tbd” (“to be determined”) into the respective text field or by setting a corresponding status. These markings can then be systematically searched for and missing information can be amended accordingly.
- *Understandable*: Requirements must be comprehensible to each stakeholder. Therefore, the type of requirements documentation (see section 4.2) can vary significantly, depending on the development phase (and therefore, depending on the involved staff). In requirements engineering, it is important to strictly define the terms used.

Fundamental principles of understandability

Along with quality criteria for requirements, there are two fundamental rules that enhance the readability of requirements:

- *Short sentences and short paragraphs:* As human short-term memory is

very limited, circumstances that belong together should be described in no more than seven sentences.

- *Formulate only one requirement per sentence:* Formulate requirements using active voice and use only one process verb. Long, complicated interlaced sentences must be avoided.

4.7 Glossary

A frequent cause for conflicts in requirements engineering is that the people that are involved in the development process have different interpretations of terms. In order to avoid these conflicts, it is necessary that everyone who is involved in the development process shares the same understanding of the terminology used. Therefore, all relevant terms must be defined in a common glossary. A glossary is a collection of term definitions and contains the following elements:

- Context-specific technical terms
- Abbreviations and acronyms
- Everyday concepts that have a special meaning in the given context
- Synonyms, i.e., different terms with the same meaning
- Homonyms, i.e., identical terms with different meanings

Consistent definitions

By defining the meaning of terms, you can increase the understandability of requirements considerably. Misunderstandings and different interpretations of terms that might lead to conflicts can be avoided from the beginning.

Reuse of glossary entries

Often, in different projects, terms are used that are similar to one another or in fact identical. This may be the case, for example, when one system is developed for different customers but within the same domain. In this case, already existing glossary entries should be reused. It may even be feasible to define such terms in a universal, inter-project glossary. The additional effort of creating such a glossary will pay off in future projects. For certain domains, collections of term definitions already exist and are publicly accessible. These may serve as the foundation for the definition of specific glossaries. For example, in [IEEE 610.12-1990], typical

terms of software engineering are defined.

Rules for Using a Glossary

Basic rules for using a glossary

Since creating a glossary is absolutely mandatory, the following must be noted:

- *The glossary must be centrally managed:* At any time, there must be only one valid glossary, which must also be centrally accessible. There must not be multiple valid glossaries.
- *Responsibility must be assigned:* One particular individual must be assigned with the task of maintaining the glossary and ensuring consistency and up-to-dateness. The necessary resources to accomplish this task must be included in the project plan.
- *The glossary must be maintained over the course of the project:* In order to ensure that the glossary is consistent and up-to-date, it must be maintained over the course of the entire project by the person that was assigned this responsibility.
- *The glossary must be commonly accessible:* The term definitions must be available for all involved personnel. This is the only way a common understanding of the terms can be ensured.
- *Using the glossary must be obligatory:* All involved personnel must be obliged to exclusively use the terms and term definitions as they have been defined in the glossary.
- *The glossary should contain the sources of the terms:* In order to be able to resolve questions and problems at any time during the course of the project, it must be possible to determine the source of a term.
- *The stakeholders should agree upon the glossary:* Only stakeholders can reliably validate the operational definitions for their respective project context. Each definition should be validated by the stakeholders or their representatives. In addition, the individual term definitions in the glossary should be explicitly approved. This approval signals that the respective term is correct and its use is obligatory.
- *The entries in the glossary should have a consistent structure:* All entries in the glossary must be structured in the same way. In order to support a consistent documentation, it is advisable to use a template for glossary

definitions. In addition to the definition and the meaning of a term, the template should specify possible synonyms and homonyms.

To reduce the effort of aligning terms with one another, it is advisable to start with the creation of the glossary early on in the project.

4.8 Summary

The documentation of requirements plays a central role in requirements engineering. As the amount of requirements is often vast, it is very important to clearly structure the requirements so that personnel not involved with the project also understand them. Also, looking up and changing requirements is simplified and accelerated in this way. This makes meeting the quality criteria for requirements documents much easier. Using customized documentation structures has proven to be suitable for that purpose. These are completed by inserting project-specific requirements written in natural language in conjunction with conceptual requirements models.

1. This section can also be treated as an appendix to the document.
2. This section can also be treated as an appendix to the document.
3. Strictly speaking, this statement holds true only for the requirements document of the next system release (see section 8.5.3).

5 Documenting Requirements in Natural Language

Elicited requirements for the system to be developed are frequently documented using natural language. Natural language has the advantage that it (allegedly) does not require preparation time in order to be read and understood by stakeholders [Robertson and Robertson 2006]. In addition, natural language is universal in the sense that it can be used to describe any circumstances. However, there are some problems associated with the use of natural language for requirements documentation.

5.1 Effects of Natural Language

Subjective perception

As natural language is inherently ambiguous and statements in natural language can often be interpreted in multiple ways, it is necessary to place special emphasis on potential ambiguities in such statements to satisfy the criterion of unambiguousness. Requirements are defined and read by people with different knowledge, different social backgrounds, and different experiences. The diversity among the people involved in the development processes may lead to misunderstanding as humans interpret information differently (they form a so-called “deep structure” in their mind) and thus construe it differently as well (e.g., as a requirement). During such a process (i.e., perception and representation of information), so-called “transformational effects” occur that show different characteristics with every human but may occur in all humans [Bandler and Grinder 1975, Bandler 1994].

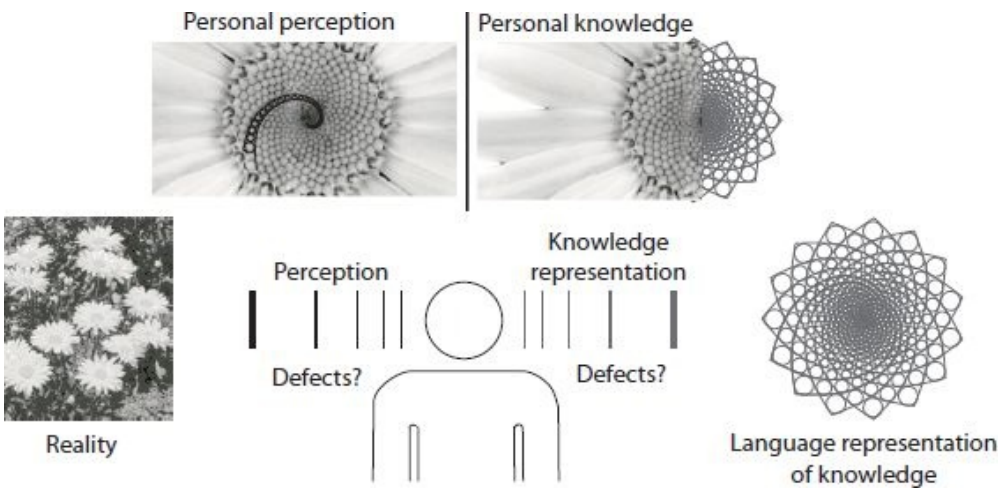


Figure 5-1 *Transformational effects in perception and representation of knowledge*

Transformational effects

The fact that transformational effects adhere to certain rules can be exploited by the requirements engineer to elicit the deep structure (i.e., what the author of a requirement really meant) from its surface structure (i.e., the requirements). The following list includes the five transformational processes that are most relevant for requirements engineering:

- Nominalization
- Nouns without reference index
- Universal quantifiers
- Incompletely specified conditions
- Incompletely specified process verbs

5.1.1 Nominalization

Reduction of processes

By means of nominalization, a (sometimes long-lasting) process is converted into a (singular) event. All information necessary to accurately describe the process is thereby lost. The process word *transmit* turns into the noun *transmission*. Other typical examples of nominalization are the terms *input, booking*, and *acceptance*.

Example 5-1: *Nominalization*

"In case of a system crash, a restart of the system shall be performed."
The terms *system crash* and *restart* each describe a process that ought to be analyzed more precisely.

Define processes completely.

Per se, there are no arguments against the use of nominalized terms to describe complex processes. However, the process should be explicitly defined by the term used. The definition of a nominalized term must not allow for any leeway in the interpretation of the processes and must precisely depict the process, including any exceptions that may occur as well as all input and output parameters. It is therefore not necessary to avoid nominalizations, but they should only be used if the underlying process is completely defined. During the linguistic analysis of a text, all nominalizations ought to be examined to determine whether they have been defined in sufficient detail in another part of the requirements document and whether they are clear for all stakeholders. If this is not the case, another requirement or a glossary entry must be created.

5.1.2 Nouns without Reference Index

Nouns with missing reference

As with process verbs, nouns are frequently incompletely specified. Linguists call this a missing or inadequate index of reference. Examples of terms that contain incompletely specified nouns are *the user, the controller, the system, the message, the data*, or *the function*.

Example 5-2: *Nouns without reference indices*

The data shall be displayed to the user on the terminal.

The following questions arise: What data exactly? Which user exactly? Which terminal exactly? If this information is amended, the requirement might thus read as follows:

Example 5-3: *Nouns with added reference indices*

The system shall display the billing data to the registered user on the terminal she is logged in to.

5.1.3 Universal Quantifiers

Specify amounts and frequencies.

Universal quantifiers specify amounts or frequencies. They group a set of objects and make a statement about the behavior of this set. When using universal quantifiers, there is the risk that the specified behavior or property does not apply to all objects within the specified set. Stakeholders tend to group objects together, even though some of these objects might be special cases or exceptions, where the behavior specified does not apply to all the objects of a group.

Identify universal quantifiers.

It must be verified whether the specified behavior really applies to all objects summarized through the quantifiers. Universal quantifiers can be easily identified through trigger words such as *never, always, no, none, every, all, some*, or *nothing*.

Example 5-4: *Universal quantifiers*

The system shall show all data sets in every submenu.

In this case, the following question must be asked: Really in every submenu? Really all data sets?

5.1.4 Incompletely Specified Conditions

Identify and clarify condition structures.

Incompletely specified conditions are another indicator of a potential loss of information. Requirements that contain conditions specify the behavior that must

occur when the condition is met. In addition, they must specify what behavior must occur if the condition is not met (the part that is often missing). Especially in complex conditional structures, decision tables can be invaluable tools to find unspecified variants of conditions or actions. Trigger words are, for instance, *if . . . then, in case, whether*, and *depending on*.

Example 5-5: *Incompletely specified condition*

The restaurant system shall offer all beverages to a registered guest over the age of 20 years.

At least one aspect remains unspecified in the example above: Which beverages shall be offered to a guest that is 20 years or younger? Clarifying this question may lead to extending the requirement as follows:

Example 5-6: *More completely specified condition*

The restaurant system shall offer

All alcohol-free beverages to any registered user younger than 21 years

All beverages including all alcoholic beverages to any user over the age of 20

5.1.5 Incompletely Specified Process Verbs

Completing process words

Some process verbs require more than one noun to be considered completely specified. The verb *transmit*, for instance, requires at least three supplements to be considered complete: *what* is being transmitted, *from where* it is being transmitted, and *to where* it is being transmitted. The feel for language (also referred to as "Sprachgefühl") is a valuable tool to help gauge which process word must be supplemented in order to be considered complete. Similarly, adjectives and adverbs may need to be supplemented as well. While the effect is much less frequent with these types of words than with verbs, it is often hard to recognize.

Avoid passive voice.

The use of incompletely specified process words can mostly be avoided or kept to a minimum if requirements are formulated using the active voice rather than the passive voice.

Example 5-7: *Requirement using the passive voice*

To log a user in, the login data is entered.

Use active voice.

In this requirement using passive voice, it is unclear who enters the login data. It is also unclear where and how this is done. If this requirement is reformulated using the active voice, at least the agent or person responsible must be included.

The same requirement using active voice might be as follows:

Example 5-8: *Requirement using active voice*

The system must allow the user to enter his user name and password using the keyboard of the terminal.

5.2 Requirement Construction using Templates

Quality by means of requirements templates and glossaries

Requirements templates provide a simple and easily understandable approach to reduce language effects when documenting requirements. Templates support the author in achieving high quality and syntactic unambiguousness in optimal time and at low costs.

Definition 5-1: *Requirements Template*

A requirements template is a blueprint for the syntactic structure of individual requirements.

In order to achieve lexical clearness in the documentation as well, it is wise to use requirements templates in conjunction with project glossaries (see section 4.7).

The following is a step-by-step description of the correct application of

requirements templates.

Step 1: Determine the Legal Obligation

How legally binding is a requirement?

In the beginning, you should determine the degree of legal obligation for a requirement. Usually, one distinguishes between legally obligatory requirements, urgently recommended requirements, future requirements, and desirable requirements. To achieve this within a requirement, you can use the modal verbs *shall, should, will*, and *may*. Alternatively, the legal obligation of a requirement can be documented by a specific requirements attribute.

Step 2: The Requirement Core

Determine the required process.

The core of each requirement is the functionality that it specifies (e.g., print, save, paste, or calculate). This functionality is referred to as the *process*. Processes are activities and may only be described using verbs. The process that depicts the system behavior by means of a requirement is to be described in step 2.

Since process words determine semantics, they must be defined as clearly as possible and be used as consistently as possible (see section 4.7).

Step 3: Characterize the Activity of a System

For functional requirements, the system activity can be classified as one of three relevant types:

- *Autonomous system activity:* The system performs the process autonomously.
- *User interaction:* The system provides the process as a service for the user.
- *Interface requirement:* The system performs a process depending on a third party (e.g., another system). The system is passive and waits for an external event.

In step 3, any kind of system activity that is specified by a requirement of the system is documented using exactly one of three requirements templates. These requirements templates are described in more detail in the following sections.

After performing steps 1 through 3, the structure of the requirement has been developed (see figure 5-2). The words that are written in angle brackets must be replaced accordingly.

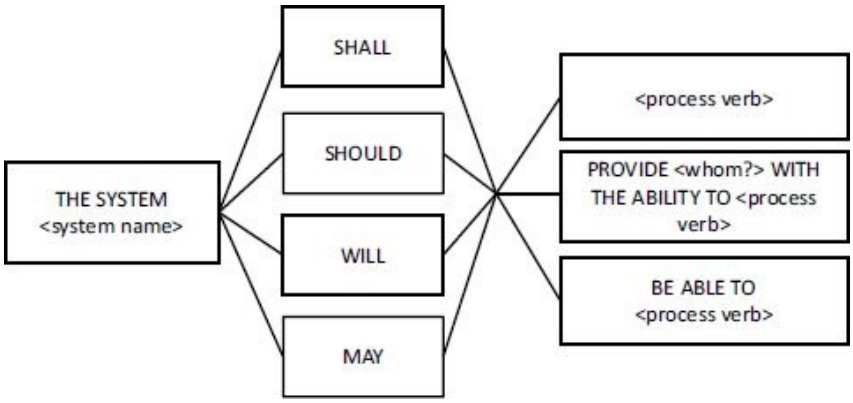


Figure 5-2 *The core of a requirement and its legal obligation*

Type 1: Autonomous system activity

The first template type is used when requirements are constructed that depict system activities that are performed autonomously. The user does not interact with the activity. We define the following requirements template:

THE SYSTEM SHALL/SHOULD/WILL/MAY <process verb>

<Process verb> depicts a process verb as described in step 2, e.g., *print* for print functionality or *calculate* for some calculation that is performed by the system.

Type 2: User interaction

If the system provides a functionality to a user (for example, by means of an input interface), or the system directly interacts with a user, requirements are constructed using template type 2:

THE SYSTEM SHALL/SHOULD/WILL/MAY provide <whom?> with the ability to <process verb>

The user that interacts with the system is integrated into the requirement through <whom?>.

Type 3:
Interface requirement

If the system performs an activity and is dependent on neighboring systems, the third template type is to be used. Whenever messages or data are received from a neighboring system, the system must react by executing specific behavior. The following template has proven itself as well suited:

THE SYSTEM SHALL/SHOULD/WILL/MAY be able to <process verb>

Step 4: Insert Objects

Complete process verbs.

Some process verbs require one or more additional objects to be considered complete (see section 5.1.5). In step 4, potentially missing objects and supplements of objects (adverbials) are identified and added to the requirement. For instance, the requirements template for the process verb *print* is amended by the information of *what* is being printed and *where* it is printed. The amendment can be seen in figure 5-3.

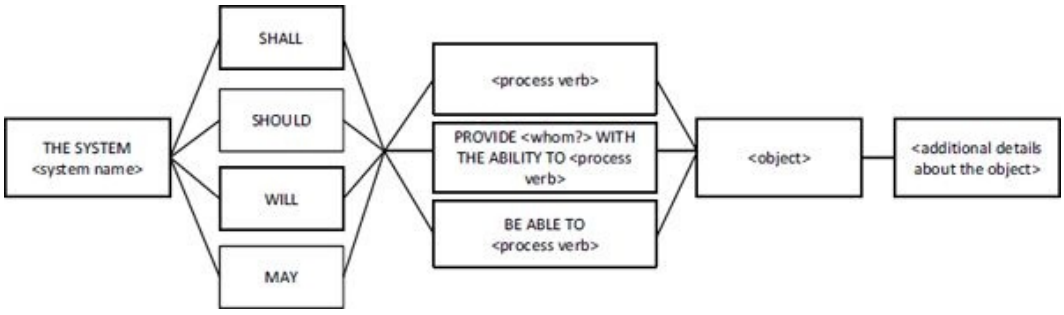


Figure 5-3 *Principle of a complete requirements template without conditions*

Step 5: Determine Logical and Temporal Conditions

Add conditions.

Typically, requirements do not document continuous functionalities, but functionalities that are performed or provided only under certain logical or temporal constraints. In order to easily differentiate between logical and temporal conditions, we choose the temporal conjunction *as soon as* for temporal conditions and the conditional conjunction *if* for logical conditions. The conjunction *when* makes not clear whether a temporal or a logical condition is described and should therefore be avoided. In step 5, quality requirements that describe the conditions under which a requirement is fulfilled are added to the beginning of a requirement as a subordinate clause.

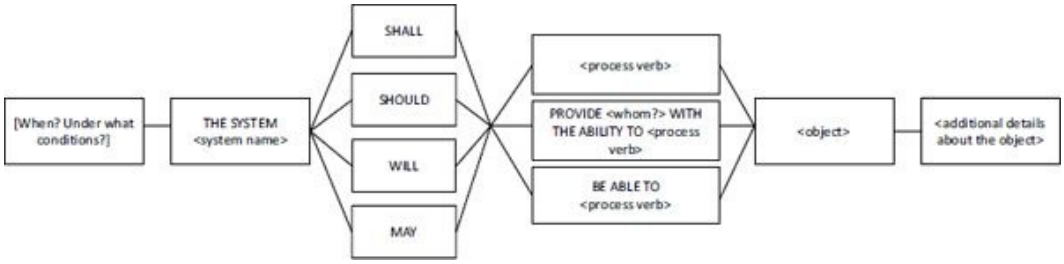


Figure 5-4 *The complete requirements template with conditions*

Requirements templates should be used when project members show interest in a formal development process. Style and creativity are harshly limited when requirements templates are used. Experience shows it is best not to make the use of requirements templates compulsory, but to offer training on the method and treat it as a supplemental tool.

5.3 Summary

System requirements are frequently documented using natural language. Typical advantages that arise from natural language requirements are good readability of requirements, the fact that natural language can be universally applied to document any circumstance, and the fact that no prior knowledge is necessary regarding the notation. On the other hand, there are a number of disadvantages that arise from the fact that natural language requirements are not formalized, e.g., ambiguity. Since project members interpret requirements differently due to differences in

their respective knowledge, social background, and experiences, using natural language for requirements documentation often leads to misunderstanding in practice. These disadvantages can be minimized during requirements documentation—for example, by making use of requirements templates and by checking the requirements against linguistic effects.